

2026년도 전국기능경기대회 채점기준

1. 채점상의 유의사항	직 종 명	클라우드컴퓨팅
1. 각 모듈의 리소스는 해당 모듈 요구사항에 명시된 Region에 생성한다. 2. 웹 페이지 접근은 Chrome 또는 Firefox로 진행한다. 3. Shell 명령어의 출력은 AWS CLI 버전에 따라 일부 다를 수 있다. 4. 문제지와 채점지에 있는 <비번호>, <자율> 등은 선수 제출 값으로 확인한다. 5. 채점은 문항 순서대로 진행한다. 6. 부분 점수가 있는 문항은 채점 항목에 표시된 배점만 인정한다. 7. 부분 점수가 따로 없는 문항은 조건을 모두 만족해야 점수로 인정한다. 8. 리소스 정보를 읽어오는 항목은 CloudShell에서 자동화 채점 스크립트로 조회하고, 채점위원은 출력 결과를 기준으로 판정한다. 9. 선수 이의가 있는 경우 채점위원이 동일 명령을 직접 실행하여 확인할 수 있다. 10. IAM User Access Key를 별도로 생성하여 채점하지 않는다. 11. CloudShell을 실행한 IAM User 또는 Role 권한으로 채점한다. 12. 채점 시 대기가 필요한 경우 항목당 최대 3분을 초과하지 않는다.		

2. 채점기준표

1) 주요항목별 배점			직 종 명		클라우드컴퓨팅			
과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제2과제	1	DocumentDB based NoSQL Application	7.5	0			0	1모듈
	2	Simplify Service Networking with VPC Lattice	7.5	0			0	2모듈
	3	Cloud Event Handling	7.5	0			0	3모듈
	4	Event-driven Pod Scaling with AWS SQS	7.5	0			0	4모듈
합계			30					

2) 채점방법 및 기준

과제 번호	일련 번호	주요항목	일련 번호	세부항목(채점방법)	배 점
제2과제	1	Document based NoSQL Application	1	DocumentDB Cluster 및 Instance	1.5
			2	Secret 및 Client EC2	1.5
			3	Client Application 및 데이터 적재 상태	1.5
			4	DocumentDB Index 및 TTL 구성	1.5
			5	NoSQL 조회 기능 검증	1.5
	2	Simplify Service Networking with VPC Lattice	1	VPC	1.5
			2	Client/Service EC2 및 애플리케이션	1.5
			3	VPC Lattice Service Network 및 Service	1.5
			4	Target Group, Listener, Security Group	1.5
			5	End-to-End 기능 검증	1.5
	3	Cloud Event Handling	1	VPC, EC2, Security Group	1.5
			2	Security Group	1.5
			3	SNS Topic 및 Lambda	1.5
			4	CloudTrail, EventBridge Rule 및 Target	1.5
			5	최종 기능 검증	1.5
	4	Event-driven Pod Scaling with AWS SQS	1	EKS Cluster, VPC, Fargate Profile	1.2 5
			2	SQS Queue, IAM, SA	1.2 5
			3	KEDA/Karpenter	1.2 5
			4	Worker Application / KEDA ScaledObject	1.2 5
			5	Karpenter NodePool, EC2NodeClass, Worker EC2	1.2 5
			6	SQS 기반 Scale Out 및 최종 기능 검증	1.2 5

합계	30
----	----

3) 채점 내용

순번	채점 항목
0	1) CloudShell에서 2과제 채점에 필요한 기본 도구를 확인합니다. 2) aws, curl, jq, kubectl 명령이 실행 가능한지 확인합니다. 3) kubectl이 없는 경우 아래 명령어로 EKS Cluster 버전에 맞는 kubectl을 설치 후 다시 확인합니다. 4) 본 항목은 채점 준비 절차이며 배점은 없습니다. <pre>aws --version curl --version jq --version if command -v kubectl >/dev/null 2>&1; then kubectl version --client else EKS_VERSION=\$(aws eks describe-cluster --region us-west-2 --name skills-sqs-cluster --query 'cluster.version' --output text) KUBECTL_VERSION="v\${EKS_VERSION}.0" curl -L -o /tmp/kubectl "https://dl.k8s.io/release/\${KUBECTL_VERSION}/bin/linux/amd64/kubectl" chmod +x /tmp/kubectl export PATH="/tmp:\$PATH" kubectl version --client fi</pre> ※ kubectl 설치 명령은 CloudShell에 kubectl이 없을 때만 실행합니다. ※ EKS Cluster가 아직 생성되지 않은 경우 4모듈 채점 직전에 본 설치 절차를 수행합니다. ※ 채점 스크립트는 kubectl을 자동 설치하지 않으므로, 4모듈 채점 전 본 준비 절차를 완료해야 합니다. 5) 아래 명령어를 입력하여 채점에 필요한 값을 변수로 선언합니다. 6) 값이 비어 있거나 None이면 해당 리소스가 요구 이름으로 생성되지 않은 것입니다. <pre>export NOSQL_CLIENT_EC2_PUBLIC_IP=\$(aws ec2 describe-instances --region ap-northeast-2 --filters Name=tag:Name,Values=skills-nosql-client-ec2 Name=instance-state-name,Values=running --query "Reservations[0].Instances[0].PublicIpAddress" --output text) export LATTICE_CLIENT_EC2_PUBLIC_IP=\$(aws ec2 describe-instances --region</pre>

	<pre> ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-client-ec2 Name=instance-state-name,Values=running --query "Reservations[0].Instances[0].PublicIpAddress" --output text) export SERVICE_NETWORK_ID=\$(aws vpc-lattice list-service-networks --region ap-northeast-1 --query "items[?name=='skills-lattice-sn'].id [0]" --output text) export TARGET_GROUP_ID=\$(aws vpc-lattice list-target-groups --region ap-northeast-1 --query "items[?name=='skills-lattice-order-tg'].id [0]" --output text) export SERVICE_ID=\$(aws vpc-lattice list-services --region ap-northeast-1 --query "items[?name=='skills-lattice-order-service'].id [0]" --output text) export SERVICE_EC2_SECURITY_GROUP_ID=\$(aws ec2 describe-instances --region ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-service-ec2 Name=instance-state-name,Values=running --query "Reservations[0].Instances[0].SecurityGroups[0].GroupId" --output text) export QUEUE_URL=\$(aws sqs get-queue-url --region us-west-2 --queue-name skills-sqs-queue --query QueueUrl --output text) </pre>
1-1	1) 아래 명령어를 입력합니다. <pre> aws docdb describe-db-clusters --region ap-northeast-2 --db-cluster-identifier skills-nosql-docdb-cluster --query 'DBClusters[0].{Cluster:DBClusterIdentifier,Status:Status,Engine:Engine,Version:En gineVersion,Encrypted:StorageEncrypted,KmsKeyId:KmsKeyId,BackupRetention:BackupRet entionPeriod,Endpoint:Endpoint,Port:Port}' --output table aws docdb describe-db-instances --region ap-northeast-2 --db-instance-identifier skills-nosql-docdb-instance-1 --query 'DBInstances[0].{Instance:DBInstanceIdentifier,Status:DBInstanceStatus,Class:DBIns tanceClass,Engine:Engine,Cluster:DBClusterIdentifier,AZ:AvailabilityZone}' --output table aws kms describe-key --region ap-northeast-2 --key-id alias/skills-nosql-docdb --query 'KeyMetadata.{Arn:Arn,Enabled:Enabled,KeyManager:KeyManager,KeyUsage:KeyUsage}' --output table </pre> 2) Cluster=skills-nosql-docdb-cluster, Status=available, Engine=docdb, Encrypted=True, Port=27017, BackupRetention>=1인지 확인합니다. Instance=skills-nosql-docdb-instance-1, Status=available, Class=db.t3.medium, Cluster=skills-nosql-docdb-cluster인지 확인합니다. KMS Key는 alias/skills-nosql-docdb로 조회되며 Enabled=True, KeyManager=CUSTOMER인지 확인합니다.
1-2	1) 아래 명령어를 입력합니다. <pre> aws secretsmanager describe-secret --region ap-northeast-2 --secret-id skills-nosql-docdb-secret --query '{Name:Name,ARN:ARN,KmsKeyId:KmsKeyId}' --output table aws secretsmanager get-secret-value --region ap-northeast-2 --secret-id skills-nosql-docdb-secret --query SecretString --output text jq -r '{username, </pre>

	host, password_set:(.password != null and .password != ""))}' aws ec2 describe-instances --region ap-northeast-2 --filters Name=tag:Name,Values=skills-nosql-client-ec2 Name=instance-state-name,Values=running --query 'Reservations[].Instances[].{Name:Tags[?Key=`Name`].Value [0],InstanceId:InstanceId,State:State.Name,Type:InstanceType,PublicIp:PublicIpAddress}' --output table 2) Secret Name=skills-nosql-docdb-secret이고 SecretString 출력에 username, host, password_set=true가 있어야 합니다. host는 DocumentDB Cluster Endpoint Hostname이어야 하며 Scheme 또는 Port가 포함되면 안 됩니다. Client EC2 Name=skills-nosql-client-ec2, State=running, PublicIp가 존재해야 합니다.
1-3	1) 아래 명령어를 입력합니다. <pre>curl -s -m 10 -w "Wnhttp_code=%{http_code}Wn" "http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/health" curl -s -m 10 -w "Wnhttp_code=%{http_code}Wn" "http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/admin/summary"</pre> 2) /health의 http_code=200이고 status=ok, database=skills_retail, port=27017, tls=true가 출력되어야 합니다. /v1/admin/summary의 http_code=200이고 counts.orders>=8, counts.products>=6, counts.sessions>=3이어야 합니다. dateFieldTypes에 orders.createdAt, orders.dueAt, products.updatedAt, sessions.lastSeen, sessions.expiresAt이 날짜 타입으로 표시되어야 합니다.
1-4	1) 아래 명령어를 입력합니다. <pre>curl -s -m 10 -w "Wnhttp_code=%{http_code}Wn" "http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/admin/indexes"</pre> 2) http_code=200이어야 합니다. orders에는 {orderId:1} unique, {customerId:1, createdAt:-1}, {status:1, dueAt:1} Index가 있어야 합니다. products에는 {productId:1} unique, {warehouseId:1, stock:1} Index가 있어야 합니다. sessions에는 {sessionId:1} unique, {expiresAt:1} TTL expireAfterSeconds=0, {customerId:1, lastSeen:-1} Index가 있어야 합니다. Index 이름은 채점하지 않습니다.
1-5	1) 아래 명령어를 입력합니다. <pre>curl -s -m 10 -w "Wnhttp_code=%{http_code}Wn" "http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/orders/0-1001" curl -s -m 10 -w "Wnhttp_code=%{http_code}Wn" "http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/customers/C001/orders" curl -s -m 10 -w "Wnhttp_code=%{http_code}Wn" "http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/orders/pending?from=2026-06-01T00:00:00Z&to=2026-06-08T00:00:00Z" curl -s -m 10 -w "Wnhttp_code=%{http_code}Wn" "http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/products/low-stock?warehouseId=W-A"</pre>

	2) 모든 조회 API의 http_code=200이어야 합니다. 응답 데이터는 retail_dataset.json 기준으로 다음을 만족해야 합니다. 0-1001 조회: orderId=0-1001, customerId=C001, status=PENDING, warehouseId=W-A C001 주문 조회: 0-1006, 0-1001, 0-1004 포함 기간 내 PENDING 조회: 0-1001, 0-1003, 0-1006 포함 W-A Low Stock 조회: P-BLU-003, P-RED-001 포함 및 P-GRN-002 미포함
2-1	1) 아래 명령어를 입력합니다. <pre>aws ec2 describe-vpcs --region ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-client-vpc,skills-lattice-service-vpc --query 'Vpcs[].{Name:Tags[?Key=='Name'].Value [0],VpcId:VpcId,Cidr:CidrBlock,State:State} ' --output table</pre> 2) skills-lattice-client-vpc의 Cidr=10.61.0.0/16, State=available이어야 합니다. skills-lattice-service-vpc의 Cidr=10.62.0.0/16, State=available이어야 합니다.
2-2	1) 아래 명령어를 입력합니다. <pre>aws ec2 describe-instances --region ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-client-ec2,skills-lattice-service-ec2 Name=instance-state-name,Values=running --query 'Reservations[].Instances[].{Name:Tags[?Key=='Name'].Value [0],Id:InstanceId,Type: InstanceType,PublicIp:PublicIpAddress,PrivateIp:PrivateIpAddress,State:State.Name} ' --output table</pre> LATTICE_CLIENT_EC2_PUBLIC_IP=\$(aws ec2 describe-instances --region ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-client-ec2 Name=instance-state-name,Values=running --query 'Reservations[0].Instances[0].PublicIpAddress' --output text 2>/dev/null true) <pre>curl -s -m 10 -w "Wnhttp_code=%{http_code}Wn" "http://\${LATTICE_CLIENT_EC2_PUBLIC_IP}/health"</pre> 2) skills-lattice-client-ec2와 skills-lattice-service-ec2가 State=running이어야 합니다. skills-lattice-client-ec2는 PublicIp가 존재해야 하고, skills-lattice-service-ec2는 PublicIp가 없어야 합니다. Client /health 응답의 http_code=200이고 status=ok, app=client가 포함되어야 합니다.
2-3	1) 아래 명령어를 입력합니다. <pre>SERVICE_NETWORK_ID=\$(aws vpc-lattice list-service-networks --region ap-northeast-1 --query 'items[?name=='skills-lattice-sn'].id [0]' --output text 2>/dev/null true)</pre> <pre>aws vpc-lattice list-service-networks --region ap-northeast-1 --query 'items[?name=='skills-lattice-sn'].{Name:name,Id:id,AssociatedVPCs:numberOfAssocia tedVPCs,AssociatedServices:numberOfAssociatedServices}' --output table</pre> <pre>aws vpc-lattice list-services --region ap-northeast-1 --query 'items[?name=='skills-lattice-order-service'].{Name:name,Id:id,Dns:dnsEntry.domain Name,Status:status}' --output table</pre>

	<pre> aws vpc-lattice list-service-network-vpc-associations --region ap-northeast-1 --service-network-identifier "\$SERVICE_NETWORK_ID" --query 'items[].{AssociationId:id,VpcId:vpclId,Status:status}' --output table VPC_ASSOCIATION_ID=\$(aws vpc-lattice list-service-network-vpc-associations --region ap-northeast-1 --service-network-identifier "\$SERVICE_NETWORK_ID" --query 'items[0].id' --output text 2>/dev/null true) echo "VPC_ASSOCIATION_ID=\${VPC_ASSOCIATION_ID}" aws vpc-lattice get-service-network-vpc-association --region ap-northeast-1 --service-network-vpc-association-identifier "\$VPC_ASSOCIATION_ID" --query '{AssociationId:id,VpcId:vpclId,Status:status,SecurityGroupIds:securityGroupIds}' --output table aws vpc-lattice list-service-network-service-associations --region ap-northeast-1 --service-network-identifier "\$SERVICE_NETWORK_ID" --query 'items[].{ServiceId:serviceId,Status:status,Dns:dnsEntry.domainName}' --output table 2) Service Network Name=skills-lattice-sn이 존재해야 합니다. Service Name=skills-lattice-order-service가 존재하고 Status=ACTIVE이며 Dns가 출력되어야 합니다. Client VPC Association의 Status=ACTIVE이어야 합니다. Service Association의 Status=ACTIVE이어야 합니다. </pre>
2-4	<pre> 1) 아래 명령어를 입력합니다. TARGET_GROUP_ID=\$(aws vpc-lattice list-target-groups --region ap-northeast-1 --query 'items[?name=='skills-lattice-order-tg'].id [0]' --output text 2>/dev/null true) SERVICE_ID=\$(aws vpc-lattice list-services --region ap-northeast-1 --query 'items[?name=='skills-lattice-order-service'].id [0]' --output text 2>/dev/null true) SERVICE_SG_IDS=\$(aws ec2 describe-instances --region ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-service-ec2 Name=instance-state-name,Values=running --query 'Reservations[0].Instances[0].SecurityGroups[].GroupId' --output text 2>/dev/null true) aws vpc-lattice list-target-groups --region ap-northeast-1 --query 'items[?name=='skills-lattice-order-tg'].{Name:name,Id:id,Type:type,Port:port,Prot ocol:protocol,Vpc:vpclIdentifier,Status:status}' --output table aws vpc-lattice list-targets --region ap-northeast-1 --target-group-identifier "\$TARGET_GROUP_ID" --query 'items[].{Target:id,Port:port,Status:status}' --output table aws vpc-lattice list-listeners --region ap-northeast-1 --service-identifier "\$SERVICE_ID" --query 'items[?name=='skills-lattice-http-listener'].{Name:name,Id:id,Port:port,Protocol: protocol}' --output table aws ec2 describe-security-groups --region ap-northeast-1 --group-ids </pre>

	<code>\$SERVICE_SG_IDS --query</code> <code>'SecurityGroups[].{GroupId:GroupId,GroupName:GroupName,VpcId:VpcId,Inbound:IpPermissions}' --output json</code> 2) Target Group Name=skills-lattice-order-tg, Type=INSTANCE, Protocol=HTTP, Port=8080이어야 합니다. Target은 skills-lattice-service-ec2 Instance이고 Status=HEALTHY이어야 합니다. Listener Name=skills-lattice-http-listener, Protocol=HTTP, Port=80이어야 합니다. Service EC2 Security Group의 TCP/8080 Inbound는 VPC Lattice Managed Prefix List로 제한되어야 하며 0.0.0.0/0 허용 시 미충족입니다.
2-5	1) 아래 명령어를 입력합니다. <code>LATTICE_CLIENT_EC2_PUBLIC_IP=\$(aws ec2 describe-instances --region ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-client-ec2 Name=instance-state-name,Values=running --query</code> <code>'Reservations[0].Instances[0].PublicIpAddress' --output text 2>/dev/null true)</code> <code>curl -s -m 10 -w "Wnhttp_code=%{http_code}Wn"</code> <code>"http://\${LATTICE_CLIENT_EC2_PUBLIC_IP}/v1/client/orders?id=1001 "</code> 2) http_code=200이어야 합니다. 응답 JSON에 client=ok, service.order_id=1001, service.via=vpc-lattice가 포함되어야 합니다.
3-1	1) 아래 명령어를 입력합니다. <code>aws ec2 describe-vpcs --region ap-southeast-1 --filters</code> <code>Name=tag:Name,Values=skills-ceh-vpc --query</code> <code>'Vpcs[].{Name:Tags[?Key=='Name'].Value [0],VpcId:VpcId,Cidr:CidrBlock}' --output table</code> <code>aws ec2 describe-instances --region ap-southeast-1 --filters</code> <code>Name=tag:Name,Values=skills-ceh-ec2 Name=instance-state-name,Values=running --query</code> <code>'Reservations[].Instances[].{Name:Tags[?Key=='Name'].Value [0],Id:InstanceId,Type:InstanceType,State:State.Name,SecurityGroups:SecurityGroups[].GroupId}' --output table</code> <code>aws ec2 describe-security-groups --region ap-southeast-1 --filters</code> <code>Name=tag:Name,Values=skills-ceh-protected-sg --query</code> <code>'SecurityGroups[].{Name:Tags[?Key=='Name'].Value [0],GroupId:GroupId,GroupName:GroupName,VpcId:VpcId}' --output table</code> 2) VPC Name=skills-ceh-vpc, Cidr=10.73.0.0/16이어야 합니다. EC2 Name=skills-ceh-ec2, State=running이어야 합니다. Security Group Name=skills-ceh-protected-sg가 존재하고 EC2에 연결되어 있어야 합니다.
3-2	1) 아래 명령어를 입력합니다. <code>aws ec2 describe-security-groups --region ap-southeast-1 --filters</code> <code>Name=tag:Name,Values=skills-ceh-protected-sg --query</code> <code>'SecurityGroups[].{GroupId:GroupId,Inbound:IpPermissions,Outbound:IpPermissionsEgr</code>

	ess}' --output json 2) Inbound 또는 IpPermissions가 빈 배열 []이어야 합니다.
3-3	1) 아래 명령어를 입력합니다. <pre>aws sns list-topics --region ap-southeast-1 --query 'Topics[?contains(TopicArn, `:skills-ceh-alert-topic`)].TopicArn' --output table</pre> <pre>aws lambda get-function-configuration --region ap-southeast-1 --function-name skills-ceh-remediate-fn --query '{FunctionName:FunctionName,State:State,LastUpdateStatus:LastUpdateStatus,Runtime:Runtime,Handler:Handler,Timeout:Timeout,Role:Role,Environment:Environment.Variables}' --output table</pre> 2) SNS Topic ARN에 skills-ceh-alert-topic이 포함되어야 합니다. Lambda FunctionName=skills-ceh-remediate-fn, State=Active, Runtime=python3.12, Handler=remediate_security_group.lambda_handler, Timeout>=30이어야 합니다. Environment에는 PROTECTED_SECURITY_GROUP_ID와 SNS_TOPIC_ARN이 존재해야 합니다.
3-4	1) 아래 명령어를 입력합니다. <pre>aws cloudtrail get-trail-status --region ap-southeast-1 --name skills-ceh-cloudtrail --query '{IsLogging:IsLogging,LatestDeliveryTime:LatestDeliveryTime,LatestDeliveryError:LatestDeliveryError}' --output table</pre> <pre>aws events describe-rule --region ap-southeast-1 --name skills-ceh-sg-change-rule --event-bus-name default --query '{Name:Name,State:State,EventPattern:EventPattern}' --output json</pre> <pre>aws events list-targets-by-rule --region ap-southeast-1 --rule skills-ceh-sg-change-rule --event-bus-name default --query 'Targets[0].{Id:Id,Arn:Arn}' --output table</pre> <pre>aws lambda get-policy --region ap-southeast-1 --function-name skills-ceh-remediate-fn --query 'Policy' --output text</pre> 2) CloudTrail IsLogging=True이어야 합니다. EventBridge Rule Name=skills-ceh-sg-change-rule, State=ENABLED이어야 합니다. EventPattern에 AuthorizeSecurityGroupIngress와 EC2 API Call via CloudTrail 조건이 포함되어야 합니다. Target Arn은 skills-ceh-remediate-fn Lambda ARN이어야 합니다. Lambda Policy에 events.amazonaws.com 호출 권한이 포함되어야 합니다.
3-5	1) 아래 명령어를 입력합니다. <pre>export PROTECTED_SECURITY_GROUP_ID=\$(aws ec2 describe-security-groups --region ap-southeast-1 --filters Name=tag:Name,Values=skills-ceh-protected-sg --query 'SecurityGroups[0].GroupId' --output text)</pre> <pre>aws ec2 authorize-security-group-ingress --region ap-southeast-1 --group-id "\$PROTECTED_SECURITY_GROUP_ID" --protocol tcp --port 22 --cidr 0.0.0.0/0</pre> <pre>jq -n --arg sg "\$PROTECTED_SECURITY_GROUP_ID"</pre>

	<pre>'{detail:{eventName:"AuthorizeSecurityGroupIngress",requestParameters:{groupId:\$sg }}}' > /tmp/skills-ceh-remediate-event.json aws lambda invoke --region ap-southeast-1 --function-name skills-ceh-remediate-fn --cli-binary-format raw-in-base64-out --payload file:///tmp/skills-ceh-remediate-event.json /tmp/skills-ceh-remediate-output.json aws ec2 describe-security-groups --region ap-southeast-1 --group-ids "\$PROTECTED_SECURITY_GROUP_ID" --query 'SecurityGroups[0].IpPermissions' --output json aws logs describe-log-groups --region ap-southeast-1 --log-group-name-prefix /aws/lambda/skills-ceh-remediate-fn --query 'logGroups[].logGroupName' --output table</pre> 2) Lambda Invoke가 성공해야 합니다. 180초 이내 SecurityGroups[0].IpPermissions가 빈 배열 []이어야 합니다. /aws/lambda/skills-ceh-remediate-fn Log Group이 출력되어야 합니다. 본 항목은 CloudTrail 전달 지연 시간을 채점하지 않습니다.
4-1	1) 아래 명령어를 입력합니다. <pre>aws eks describe-cluster --region us-west-2 --name skills-sqs-cluster --query 'cluster.{Name:name,Status:status,Endpoint:endpoint,VpcId:resourcesVpcConfig.vpcId ,Subnets:resourcesVpcConfig.subnetIds}' --output table aws eks describe-fargate-profile --region us-west-2 --cluster-name skills-sqs-cluster --fargate-profile-name skills-sqs-fp-keda --query 'fargateProfile.{Name:fargateProfileName,Status:status,Namespaces:selectors[].name space}' --output table aws eks describe-fargate-profile --region us-west-2 --cluster-name skills-sqs-cluster --fargate-profile-name skills-sqs-fp-karpenter --query 'fargateProfile.{Name:fargateProfileName,Status:status,Namespaces:selectors[].name space}' --output table aws eks update-kubeconfig --region us-west-2 --name skills-sqs-cluster kubectl get nodes -l eks.amazonaws.com/compute-type=fargate -o json jq ' [.items[] {name:.metadata.name}]'</pre> 2) Cluster Name=skills-sqs-cluster, Status=ACTIVE, Endpoint 출력, VpcId 및 Subnets 출력이 있어야 합니다. Fargate Profile skills-sqs-fp-keda와 skills-sqs-fp-karpenter가 Status=ACTIVE이어야 합니다. skills-sqs-fp-keda Namespaces에 keda, skills-sqs-fp-karpenter Namespaces에 karpenter가 포함되어야 합니다. CloudShell에서 kubectl 접근이 가능하고 Fargate Node 이름이 출력되어야 합니다.
4-2	1) 아래 명령어를 입력합니다. <pre>QUEUE_URL=\$(aws sqs get-queue-url --region us-west-2 --queue-name skills-sqs-queue --query QueueUrl --output text 2>/dev/null true) echo "QUEUE_URL=\${QUEUE_URL}" if [-n "\$QUEUE_URL"] && ["\$QUEUE_URL" != "None"]; then</pre>

	<pre>aws sqs get-queue-attributes --region us-west-2 --queue-url "\$QUEUE_URL" --attribute-names QueueArn VisibilityTimeout --output table else echo "skills-sqs-queue Queue URL 식별 실패" fi kubectl get serviceaccount keda-operator -n keda -o jsonpath='keda/keda-operator role={.metadata.annotations.eksW.amazonawsW.com/role-arn}{ "Wn"}' kubectl get serviceaccount karpenter -n karpenter -o jsonpath='karpenter/karpenter role={.metadata.annotations.eksW.amazonawsW.com/role-arn}{ "Wn"}' kubectl get serviceaccount sqs-worker-sa -n skills-sqs -o jsonpath='skills-sqs/sqs-worker-sa role={.metadata.annotations.eksW.amazonawsW.com/role-arn}{ "Wn"}'</pre> 2) Queue URL이 출력되어야 합니다. QueueArn이 출력되고 VisibilityTimeout>=300이어야 합니다. keda/keda-operator, karpenter/karpenter, skills-sqs/sqs-worker-sa의 role-arn annotation이 비어 있지 않아야 합니다.
4-3	1) 아래 명령어를 입력합니다. <pre>kubectl get deployment keda-operator -n keda -o json jq '{name:.metadata.name, availableReplicas:(.status.availableReplicas // 0), readyReplicas:(.status.readyReplicas // 0)}'</pre> <pre>kubectl get pods -n keda -o json jq ' [.items[] select(.metadata.name test("^keda-operator-")) {name:.metadata.name, phase:.status.phase, node:.spec.nodeName}]'</pre> <pre>kubectl get deployment karpenter -n karpenter -o json jq '{name:.metadata.name, availableReplicas:(.status.availableReplicas // 0), readyReplicas:(.status.readyReplicas // 0)}'</pre> <pre>kubectl get pods -n karpenter -o json jq ' [.items[] select(.metadata.name test("^karpenter-")) {name:.metadata.name, phase:.status.phase, node:.spec.nodeName}]'</pre> 2) keda-operator Deployment의 availableReplicas가 1 이상이어야 합니다. karpenter Deployment의 availableReplicas가 1 이상이어야 합니다. 각 Deployment Pod의 phase는 Running이어야 합니다. 해당 Pod들은 Fargate Node에서 실행되어야 합니다.
4-4	1) 아래 명령어를 입력합니다. <pre>kubectl get deployment sqs-worker -n skills-sqs -o jsonpath='name={.metadata.name}{ "Wn"}serviceAccountName={.spec.template.spec.servi ceAccountName}{ "Wn"}selector={.spec.selector.matchLabels}{ "Wn"}podLabels={.spec.te mplate.metadata.labels}{ "Wn"}nodeSelector={.spec.template.spec.nodeSelector}{ "Wn"} env={.spec.template.spec.containers[0].env}{ "Wn"}image={.spec.template.spec.contai ners[0].image}{ "Wn"}'</pre> <pre>kubectl get scaledobject sqs-worker-scaledobject -n skills-sqs -o json jq '{name:.metadata.name, namespace:.metadata.namespace,</pre>

	scaleTargetRef:.spec.scaleTargetRef, minReplicaCount:.spec.minReplicaCount, maxReplicaCount:.spec.maxReplicaCount, pollingInterval:.spec.pollingInterval, cooldownPeriod:.spec.cooldownPeriod, triggers:.spec.triggers}' kubectl get triggerauthentication sqs-worker-trigger-auth -n skills-sqs -o json jq '{name:.metadata.name, namespace:.metadata.namespace, podIdentity:.spec.podIdentity}' 2) Deployment name=sqs-worker, serviceAccountName=sqs-worker-sa이어야 합니다. selector/podLabels에 app=sqs-worker가 포함되어야 합니다. nodeSelector에 karpenter.sh/nodepool=skills-sqs-nodepool, skills-nodepool=event-worker가 포함되어야 합니다. env에 SQS_QUEUE_URL, AWS_REGION, PROCESSING_SECONDS가 있어야 합니다. ScaledObject name=sqs-worker-scaledobject, scaleTargetRef.name=sqs-worker, minReplicaCount=0, maxReplicaCount=6, pollingInterval<=15, cooldownPeriod<=30이어야 합니다. Trigger type은 aws-sqs-queue이고 queueLength=2이어야 합니다. TriggerAuthentication name=sqs-worker-trigger-auth, namespace=skills-sqs가 출력되어야 합니다. podIdentity.provider=aws-eks이어야 합니다.
4-5	1) 아래 명령어를 입력합니다. kubectl get nodepool skills-sqs-nodepool -o json jq '{name:.metadata.name, labels:.spec.template.metadata.labels, nodeClassRef:.spec.template.spec.nodeClassRef, requirements:.spec.template.spec.requirements, consolidationPolicy:.spec.disruption.consolidationPolicy}' kubectl get ec2nodeclass skills-sqs-nodeclass -o json jq '{name:.metadata.name, role:.spec.role, instanceProfile:.spec.instanceProfile, subnetSelectorTerms:.spec.subnetSelectorTerms, securityGroupSelectorTerms:.spec.securityGroupSelectorTerms, amiFamily:.spec.amiFamily}' kubectl get nodes -l karpenter.sh/nodepool=skills-sqs-nodepool,skills-nodepool=event-worker -o json jq '[.items[] {name:.metadata.name, nodepool:.metadata.labels["karpenter.sh/nodepool"], skillsNodepool:.metadata.labels["skills-nodepool"], instanceType:.metadata.labels["node.kubernetes.io/instance-type"], ready:(.status.conditions[] select(.type=="Ready"))[0].status}]' kubectl get pods -n skills-sqs -l app=sqs-worker -o json jq '[.items[] {name:.metadata.name, phase:.status.phase, node:.spec.nodeName}]' 2) NodePool name=skills-sqs-nodepool이어야 합니다. NodePool name=skills-sqs-nodepool이어야 합니다. NodePool labels에 skills-nodepool=event-worker가 있어야 합니다. NodePool nodeClassRef.name=skills-sqs-nodeclass이어야 합니다. NodePool consolidationPolicy가 비어 있지 않아야 합니다.

	EC2NodeClass name=skills-sqs-nodeclass이어야 하며 role 또는 instanceProfile이 출력되어야 합니다. Worker EC2 Node는 nodepool=skills-sqs-nodepool, skillsNodepool=event-worker, ready=True가 출력되어야 합니다. Worker Pod는 phase와 node가 출력되어야 합니다. 단, Worker Deployment 및 KEDA 설정의 minReplicaCount=0 특성상 채점 시점에 Worker EC2 Node 또는 Worker Pod가 없을 수 있으므로, 이 경우 동일 과제의 4-6 SQS 기반 Scale Out 실행 결과에서 확인된 Worker EC2 Node 및 Worker Pod 출력을 포함하여 판정할 수 있습니다.
4-6	1) 아래 명령어로 SQS 메시지를 12개 발행합니다. <pre>if [-z "\$QUEUE_URL"] ["\$QUEUE_URL" = "None"]; then echo "skills-sqs-queue Queue URL 식별 실패" else SENT=0 for I in \$(seq 1 12); do aws sqs send-message --region us-west-2 --queue-url "\$QUEUE_URL" --message-body "judge-\$I" >/dev/null 2>&1 && SENT=\$((SENT + 1)) done echo "sent=\${SENT}" for T in 60 120 180; do sleep 60 echo "after_\${T}s" aws sqs get-queue-attributes --region us-west-2 --queue-url "\$QUEUE_URL" --attribute-names ApproximateNumberOfMessages ApproximateNumberOfMessagesNotVisible ApproximateNumberOfMessagesDelayed --output table kubectl get deployment sqs-worker -n skills-sqs kubectl get pods -n skills-sqs -l app=sqs-worker -o wide kubectl get nodes -l karpenter.sh/nodepool=skills-sqs-nodepool,skills-nodepool=event-worker -o wide kubectl get nodeclaims -l karpenter.sh/nodepool=skills-sqs-nodepool done fi</pre> 2) SQS 메시지 발행 수가 12개이어야 합니다. 마지막 메시지 발행 완료 후 180초 이내 sqs-worker Pod가 증가해야 합니다. 180초 이내 Karpenter EC2 Worker Node가 1개 이상 Ready 상태로 확인되어야 합니다. SQS ApproximateNumberOfMessages가 감소해야 합니다. ApproximateNumberOfMessagesNotVisible은 Worker가 처리 중인 메시지이므로, 대기 메시지가 감소하고 Pod가 처리 중이면 실패로 보지 않습니다. Scale In 완료 시간은 채점하지 않습니다. 본 항목은 SQS 메시지를 생성하므로 4모듈의 마지막 채점 항목으로 수행합니다.

